Java Practical: Abstract Classes and Interfaces - Answers

Exercise 1: Implementing an Abstract Class - Solution

```
abstract class Animal {
  String name;
  public Animal(String name) {
    this.name = name;
  }
  abstract void makeSound();
  public void displayInfo() {
    System.out.println("Animal: " + name);
  }
}

class Dog extends Animal {
  public Dog(String name) {
    super(name);
  }

  @Override
  void makeSound() {
    System.out.println(name + " barks: Woof Woof!");
  }
}

class Cat extends Animal {
  public Cat(String name) {
    super(name);
  }
```

```
  @Override
  void makeSound() {
    System.out.println(name + " meows: Meow Meow!");
  }
}

public class AbstractClassExample {
  public static void main(String[] args) {
    Animal dog = new Dog("Buddy");
    Animal cat = new Cat("Whiskers");
    dog.displayInfo();
    dog.makeSound();

    cat.displayInfo();
    cat.makeSound();
  }
}
```

Exercise 2: Implementing an Interface - Solution

```
interface Vehicle {
  void start();
  void stop();
}

class Car implements Vehicle {
  @Override
  public void start() {
    System.out.println("Car is starting...");
  }
```

```
  @Override
  public void stop() {
    System.out.println("Car is stopping...");
  }
}

class Bike implements Vehicle {
  @Override
  public void start() {
    System.out.println("Bike is starting...");
  }

  @Override
  public void stop() {
    System.out.println("Bike is stopping...");
  }
}

public class InterfaceExample {
  public static void main(String[] args) {
    Vehicle myCar = new Car();
    Vehicle myBike = new Bike();
    myCar.start();
    myCar.stop();
    myBike.start();
    myBike.stop();
  }
}
```

Exercise 3: Combining Abstract Classes and Interfaces - Solution

```
abstract class Appliance {
  String brand;
  public Appliance(String brand) {
    this.brand = brand;
  }

  abstract void showFunction();

  public void displayBrand() {
    System.out.println("Appliance Brand: " + brand);
  }
}

interface SmartDevice {
  void connectToWiFi();
}

class SmartFridge extends Appliance implements SmartDevice {
  public SmartFridge(String brand) {
    super(brand);
  }

  @Override
  void showFunction() {
    System.out.println("SmartFridge can cool food and make ice.");
  }

  @Override
    public void connectToWiFi() {
    System.out.println("SmartFridge connected to WiFi for remote
monitoring.");
```

```
  }
}

public class HybridExample {
  public static void main(String[] args) {
    SmartFridge myFridge = new SmartFridge("Samsung");
    myFridge.displayBrand();
    myFridge.showFunction();
    myFridge.connectToWiFi();
  }
}
```

Exercise 4: Multiple Inheritance Using Interfaces - Solution

```
// Interface 1
interface Flyable {
  void fly();
}

// Interface 2
interface Swimmable {
  void swim();
}

// Class implementing both interfaces
class Duck implements Flyable, Swimmable {
  @Override
  public void fly() {
    System.out.println("Duck is flying.");
  }
  @Override
  public void swim() {
```

```
    System.out.println("Duck is swimming.");
  }
}

public class MultipleInheritanceExample {
  public static void main(String[] args) {
    Duck myDuck = new Duck();
    myDuck.fly();
    myDuck.swim();
  }
}
```

Exercise 5: Smart Transport System - Solution

```
// Abstract class Transport
abstract class Transport {
  String model;
  public Transport(String model) {
    this.model = model;
  }

  abstract void move();

  public void showModel() {
    System.out.println("Transport Model: " + model);
  }
}

// Interface SelfDriving
interface SelfDriving {
  void autoPilot();
}
```

```
// AutonomousCar class
class AutonomousCar extends Transport implements SelfDriving {
  public AutonomousCar(String model) {
    super(model);
}

  @Override
  void move() {
    System.out.println(model + " is driving on the road.");
  }

  @Override
  public void autoPilot() {
    System.out.println(model + " is switching to autopilot mode.");
  }
}

// AutonomousDrone class
class AutonomousDrone extends Transport implements SelfDriving {
  public AutonomousDrone(String model) {
    super(model);
  }

  @Override
  void move() {
    System.out.println(model + " is flying in the air.");
  }

  @Override
  public void autoPilot() {
```

```
    System.out.println(model + " is navigating autonomously.");
  }
}

// Main class
public class SmartTransportSystem {
  public static void main(String[] args) {
    AutonomousCar tesla = new AutonomousCar("Tesla Model X");
    AutonomousDrone drone = new AutonomousDrone("DJI Phantom");
    tesla.showModel();
    tesla.move();
    tesla.autoPilot();
    drone.showModel();
    drone.move();
    drone.autoPilot();
  }
}
```